Doodle Pals Uploader — complete upload script and evidence

Google Cloud project 894125991209 (doodle-pals) · Channel: Doodle Pals (UCmzVOSXBeD4EOyCfn2pxTUg) · Website: https://hamedr2000.github.io/youtubeviwebot/

1. How the upload flow works

1. A private GitHub Actions workflow renders 5 original animated Shorts per day and puts them on a private review page (GitHub pre-release).
2. The channel owner (the only user) watches the videos and approves or rejects each one.
3. Only after approval, the workflow runs in **publish** mode and the script below uploads the approved videos with `videos.insert` (madeForKids = true, category 27 Education). `channels.list(mine=true)` is used only once at authorization to confirm the channel.
4. OAuth credentials (client ID/secret, refresh token for the owner's own account) are stored as encrypted GitHub Actions secrets.

2. Evidence: first end-to-end run (10 November 2026)

Video	YouTube URL	Status
circle-cookie	https://youtube.com/shorts/cB4B6dtss9w	Private (pending audit)
diamond-kite	https://youtube.com/shorts/WrKRIzcPZbo	Private (pending audit)
heart-strawberry	https://youtube.com/shorts/mvmjiCchPK4	Private (pending audit)
oval-avocado	https://youtube.com/shorts/kk0jTLQcrFs	Private (pending audit)
rectangle-door	https://youtube.com/shorts/gWuwEosQbyM	Private (pending audit)

Uploaded through this exact script; currently locked to private until the API audit is approved.

3. Source code (public)

Upload script: https://github.com/HamedR2000/youtubeviwebot/blob/main/kids-shorts/upload.py
Workflow: https://github.com/HamedR2000/youtubeviwebot/blob/main/.github/workflows/kids_shorts.yml
Authorization helper: https://github.com/HamedR2000/youtubeviwebot/blob/main/kids-shorts/get_token.py

4. Complete upload script — kids-shorts/upload.py

```
"""Upload an approved daily batch to YouTube.

Usage: python3 upload.py <batch_dir> [--skip id1,id2] [--privacy public|unlisted|private] [--dry-run]
Reads <batch_dir>/batch.json (written by daily.js) and uploads each video as Made-for-Kids, Education.

Credentials (env): YT_CLIENT_ID, YT_CLIENT_SECRET, YT_REFRESH_TOKEN  (create once with get_token.py)
"""
```

```
import argparse
import json
import os
import sys

p = argparse.ArgumentParser()
p.add_argument('batch_dir')
p.add_argument('--skip', default='')
p.add_argument('--privacy', default='public', choices=['public', 'unlisted', 'private'])
p.add_argument('--dry-run', action='store_true')
args = p.parse_args()

batch = json.load(open(os.path.join(args.batch_dir, 'batch.json')))
skip = {s.strip() for s in args.skip.split(',') if s.strip()}
videos = [v for v in batch['videos'] if v['id'] not in skip]
print(f"{batch['date']}: uploading {len(videos)} video(s), skipping {sorted(skip) or 'none'}")

if args.dry_run:
    for v in videos:
        print('  would upload', v['file'], '-', v['title'])
    sys.exit(0)

from google.oauth2.credentials import Credentials  # noqa: E402
from googleapiclient.discovery import build  # noqa: E402
from googleapiclient.http import MediaFileUpload  # noqa: E402

creds = Credentials(
    token=None,
    refresh_token=os.environ['YT_REFRESH_TOKEN'],
    client_id=os.environ['YT_CLIENT_ID'],
    client_secret=os.environ['YT_CLIENT_SECRET'],
    token_uri='https://oauth2.googleapis.com/token',
    scopes=['https://www.googleapis.com/auth/youtube.upload',
'https://www.googleapis.com/auth/youtube.readonly'],
)
yt = build('youtube', 'v3', credentials=creds, cache_discovery=False)

results = []
for v in videos:
    body = {
        'snippet': {
            'title': v['title'][:100],
            'description': v['description'],
            'tags': v['tags'],
            'categoryId': v.get('categoryId', '27'),
            'defaultLanguage': 'en',
            'defaultAudioLanguage': 'en',
        },
        'status': {
            'privacyStatus': args.privacy,
            'selfDeclaredMadeForKids': bool(v.get('madeForKids', True)),
            'containsSyntheticMedia': False,
        },
    }
    media = MediaFileUpload(os.path.join(args.batch_dir, v['file']), mimetype='video/mp4',
resumable=True)
    req = yt.videos().insert(part='snippet,status', body=body, media_body=media)
    resp = None
    while resp is None:
        _, resp = req.next_chunk()
    url = f"https://youtube.com/shorts/{resp['id']}"
    print('  uploaded', v['id'], '->', url)
    results.append({'id': v['id'], 'youtube_id': resp['id'], 'url': url})
```

```
json.dump(results, open(os.path.join(args.batch_dir, 'uploaded.json'), 'w'), indent=2)
```

5. Publish job — .github/workflows/kids_shorts.yml

```
  publish:
    if: inputs.mode == 'publish'
    runs-on: ubuntu-latest
    timeout-minutes: 30
    defaults:
      run:
        working-directory: kids-shorts
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install google-api-python-client google-auth
      - name: Download approved batch
        env:
          GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}
        run: |
          test -n "${{ inputs.date }}" || { echo "date is required for publish"; exit 1; }
          gh release download "shorts-${{ inputs.date }}" --dir batch
      - name: Upload to YouTube
        env:
          YT_CLIENT_ID: ${{ secrets.YT_CLIENT_ID }}
          YT_CLIENT_SECRET: ${{ secrets.YT_CLIENT_SECRET }}
          YT_REFRESH_TOKEN: ${{ secrets.YT_REFRESH_TOKEN }}
        run: python3 upload.py batch --skip "${{ inputs.skip }}" --privacy "${{ inputs.privacy }}"
      - name: Mark Release as published
        env:
          GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}
        run: |
          LINKS=$(python3 -c "import json;print('\n'.join('- '+r['id']+': '+r['url'] for r in
json.load(open('batch/uploaded.json'))))")
          gh release upload "shorts-${{ inputs.date }}" batch/uploaded.json --clobber
          gh release edit "shorts-${{ inputs.date }}" --prerelease=false --notes "Published to YouTube.

          $LINKS"
```